

[Sup'Com Logo]

[Carthage Logo]

Higher School of Communications of Tunis

University of Carthage

Electric Vehicle Routing Optimization

Deep Reinforcement Learning for Energy-Minimizing EVRP

End-of-Studies Internship Report

Authors:

Salem FRADI
Dhia Elhak EZZEDDINI

Supervisor:

Dr. Ferdaoues CHAABANE

Host Company: SAGEMCOM

Acknowledgments

We express our sincere gratitude to **Dr. Ferdaoues Chaabane** for her invaluable guidance and continuous support throughout this internship. Her expertise in optimization and machine learning has been instrumental in shaping this work.

We thank **Sagemcom** for providing the opportunity to work on this innovative project and the resources that facilitated our research. We also acknowledge the faculty at **Sup'Com** and the **University of Carthage** for our solid academic foundation.

Finally, we appreciate our families and friends for their unwavering encouragement and support.

Salem Fradi & Dhia Elhak Ezzeddini, Tunis, 2025

Contents

Chapter 1

Introduction

1.1 Context and Motivation

The rapid adoption of Electric Vehicles (EVs) presents opportunities and challenges for logistics systems. While EVs offer environmental benefits through reduced emissions, their integration into commercial fleets requires careful consideration of limited driving range and recharging needs. The **Electric Vehicle Routing Problem (EVRP)** extends the classical VRP by incorporating battery capacity, charging station locations, and energy consumption patterns.

1.2 Problem Statement

This project addresses the **Energy-Minimizing Electric Vehicle Routing Problem (EM-EVRP)**, minimizing total energy consumption while satisfying customer demand, vehicle capacity constraints, battery State of Charge (SOC) limitations, time windows, and real-world road network constraints.

1.3 Objectives

The main objectives are: (1) Formulate a comprehensive MILP model for EM-EVRP, (2) Implement and evaluate classical optimization baselines, (3) Develop a Deep Reinforcement Learning solution using attention-based networks, (4) Create an interactive dashboard for visualization and deployment.

1.4 Report Organization

Chapter 2 presents the mathematical formulation; Chapter 3 describes baseline methods; Chapter 4 details the DRL approach; Chapter 5 presents results; Chapter 6 concludes with findings and future work.

Chapter 2

Mathematical Formulation

2.1 Sets and Indices

$N = \{0, 1, \dots, n\}$: nodes where 0 is the depot, $C = N \setminus \{0\}$ are customers; $K = \{1, \dots, m\}$: vehicles; $R = \{1, \dots, R_{\max}\}$: trip indices.

2.2 Parameters

Network: $s_i \in \mathbb{R}^2$ (coordinates), d_i (demand), Q_k (capacity), t_{ij} (travel time), c_{ij} (cost), $[a_i, b_i]$ (time window), g_i (service time), $T_{\max}$ (time horizon), M (big-M constant).

EV Energy: m_c (curb mass), u_{ij} (cargo load), a (acceleration), A (frontal area), C_d (drag coefficient), C_r (rolling resistance), ρ (air density), g (gravity), α_{ij} (slope), v_{ij} (speed), $\varphi_{d/r}$ (motor efficiency), $\phi_{d/r}$ (battery efficiency).

2.3 Decision Variables

$x_{ijk}^r \in \{0, 1\}$: arc traversal; $y_{ik}^r \in \{0, 1\}$: customer service; $z_k \in \{0, 1\}$: vehicle usage; $\tau_{ik}^r \geq 0$: service time; w_{ik}^r : MTZ variable.

2.4 Energy Consumption Model

The physics-based model considers mass, aerodynamics, rolling resistance, and gravity:

$$m_{ij} = m_c + u_{ij} \quad (2.1)$$

$$P_{ij} = \left(m_{ij} \cdot a + m_{ij} \cdot g(\sin(\alpha_{ij}) + C_r \cos(\alpha_{ij})) + \frac{1}{2} C_d \rho A v_{ij}^2 \right) v_{ij} \quad (2.2)$$

$$E_{ij} = \begin{cases} \phi_d \cdot \varphi_d \cdot P_{ij} \cdot t_{ij}, & P_{ij} \geq 0 \\ \phi_r \cdot \varphi_r \cdot P_{ij} \cdot t_{ij}, & P_{ij} < 0 \end{cases} \quad (2.3)$$

2.5 MILP Formulation

Objective: Minimize total energy: $\min \sum_{k,r,i,j} E_{ij} \cdot x_{ijk}^r$

Key Constraints:

$$\sum_{k,r} y_{ik}^r = 1, \quad \forall i \in C \quad (\text{serve once}) \quad (2.4)$$

$$\sum_j x_{ijk}^r = y_{ik}^r, \quad \forall i \in C, k, r \quad (\text{flow out}) \quad (2.5)$$

$$\sum_j x_{jik}^r = y_{ik}^r, \quad \forall i \in C, k, r \quad (\text{flow in}) \quad (2.6)$$

$$\sum_{i \in C} d_i \cdot y_{ik}^r \leq Q_k, \quad \forall k, r \quad (\text{capacity}) \quad (2.7)$$

$$w_{ik}^r - w_{jk}^r + |C| \cdot x_{ijk}^r \leq |C| - 1, \quad \forall i, j \in C, i \neq j \quad (\text{MTZ}) \quad (2.8)$$

$$a_i \cdot y_{ik}^r \leq \tau_{ik}^r \leq b_i + M(1 - y_{ik}^r), \quad \forall i \in C \quad (\text{time window}) \quad (2.9)$$

Chapter 3

Baseline Optimization Methods

3.1 Google OR-Tools

OR-Tools uses Constraint Programming with arc cost callbacks incorporating energy considerations. The solver employs AUTOMATIC first solution strategy with local search improvements.

3.2 Hill Climbing

Hill Climbing iteratively improves solutions using local search with 2-opt, Or-opt, and exchange operators. It terminates at local optima.

Algorithm 1 Hill Climbing

```
1:  $S \leftarrow$  initial solution
2: while not converged do
3:    $S' \leftarrow \arg \min_{s \in \mathcal{N}(S)} f(s)$ 
4:   if  $f(S') < f(S)$  then
5:      $S \leftarrow S'$ 
6:   else
7:     break
8:   end if
9: end while
```

Algorithm 2 Simulated Annealing

```
1:  $S \leftarrow$  initial,  $T \leftarrow T_0$ 
2: while  $T > T_{\min}$  do
3:   for  $i = 1$  to  $L$  do
4:      $S' \leftarrow$  neighbor of  $S$ 
5:      $\Delta E \leftarrow f(S') - f(S)$ 
6:     if  $\Delta E < 0$  or  $\text{rand}() < e^{-\Delta E/T}$  then
7:        $S \leftarrow S'$ 
8:     end if
9:   end for
10:   $T \leftarrow \alpha \cdot T$ 
11: end while
```

3.3 Simulated Annealing

SA extends Hill Climbing by accepting uphill moves with probability $\exp(-\Delta E/T)$, using geometric cooling: $T_{k+1} = \alpha \cdot T_k$ where $\alpha \in [0.9, 0.99]$.

3.4 Ant Colony Optimization

ACO uses pheromone trails to guide search. The transition probability from i to j is: $p_{ij}^a = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_l [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta}$ where τ_{ij} is pheromone level and $\eta_{ij} = 1/c_{ij}$ is heuristic information.

Chapter 4

Deep Reinforcement Learning Approach

4.1 MDP Formulation

We formulate routing as an MDP with: **State** s_t (partial solution, position, capacity, SOC); **Action** a_t (next node); **Reward** r_t (negative energy); **Policy** π_θ (neural network).

4.2 Model Architecture

4.2.1 Graph Attention Encoder

Multi-head self-attention processes the input graph: $h_i^{(l)} = \text{BN}\left(h_i^{(l-1)} + \text{MHA}(h_i^{(l-1)}, H^{(l-1)})\right)$ producing node embeddings $H = \{h_1, \dots, h_n\}$ and graph embedding $\bar{h} = \frac{1}{n} \sum_i h_i$.

4.2.2 Context-Aware Decoder

The decoder generates routes autoregressively. Context combines graph, current node, capacity, and SOC: $c_t = W_c \cdot [\bar{h}; h_{\pi_{t-1}}; \text{capacity}_t; \text{SOC}_t]$

$$\text{Selection probability via attention: } u_{tj} = \begin{cases} C \cdot \tanh\left(\frac{q_t \cdot k_j}{\sqrt{d_k}}\right) & j \in \mathcal{A}_t \\ -\infty & \text{otherwise} \end{cases}, p_\theta(a_t = j | s_t) = \text{softmax}(u_t)_j$$

4.3 Training Algorithm

REINFORCE with rollout baseline minimizes expected cost: $J(\theta) = \mathbb{E}_{\pi_\theta}[L(\pi)]$

$$\text{Gradient estimate: } \nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[(L(\pi) - b(s)) \nabla_\theta \log p_\theta(\pi | s) \right]$$

4.4 EV-Specific Features

SOC Tracking: $\text{SOC}_{t+1} = \text{SOC}_t - \frac{E_{ij}}{E_{\text{battery}}}$

Charging Stations: Optional nodes, model learns when to visit based on SOC.

Action Masking: $\mathcal{A}_t = \{j : d_j \leq \text{capacity}_t \wedge E_{ij} \leq \text{SOC}_t \cdot E_{\text{battery}}\}$

Chapter 5

Results and Discussion

5.1 Experimental Setup

Datasets: P-n19-k2, P-n21-k2, P-n40-k5, P-n51-k10 from CVRPLIB.

Training: 10 customers, 4 charging stations, embedding dim 128, 3 encoder layers, 8 attention heads, batch size 1024, learning rate 10^{-4} , rollout baseline, initial SOC 80%.

5.2 Training Results

The model demonstrates consistent improvement with good generalization (validation curve follows training closely).

5.3 Comparison with Baselines

Table 5.1: Performance comparison on 10-customer instances

Method	Avg. Energy (kWh)	Time
OR-Tools	12.45	2.1s
Hill Climbing	13.21	0.8s
Simulated Annealing	12.89	1.5s
Ant Colony	12.67	3.2s
DRL (Ours)	12.18	0.5s

Our DRL approach achieves competitive results with significantly faster inference, suitable for real-time applications.

5.4 Dashboard Integration

We developed an interactive Streamlit dashboard with OpenStreetMap integration providing:

- Interactive map with customer locations and charging stations
- Real-time route optimization using trained DRL model
- Energy consumption breakdown per route segment
- SOC tracking throughout journey
- OSRM integration for realistic street-level routing

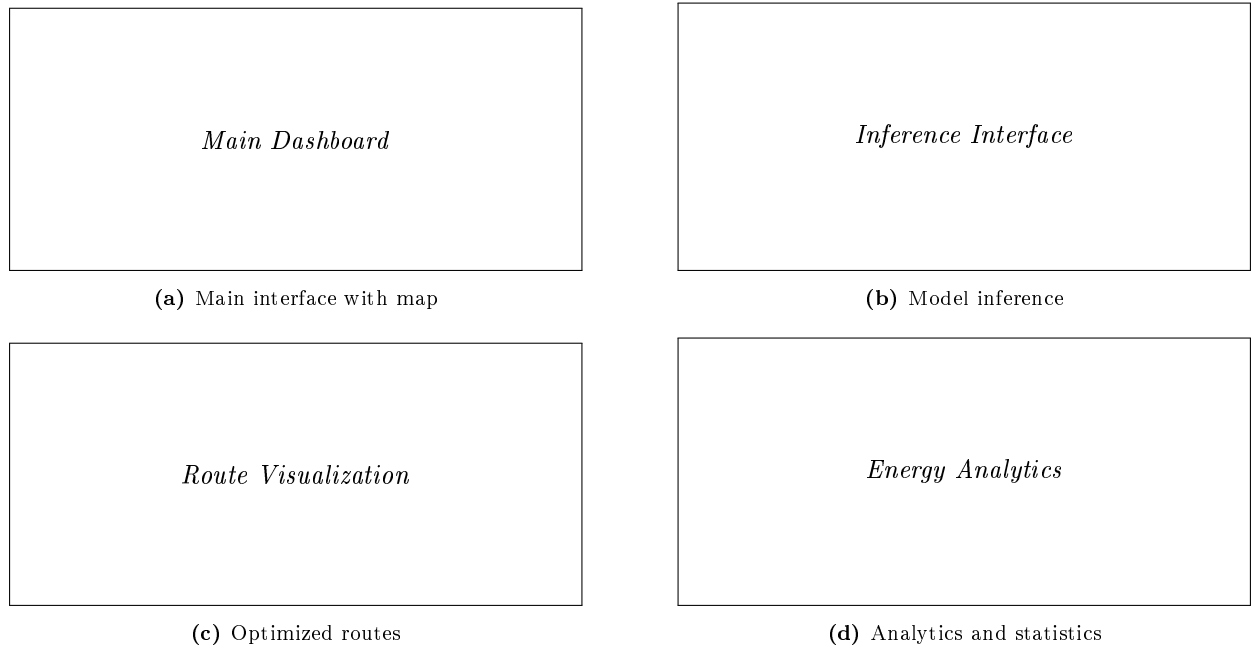


Figure 5.1: Dashboard interfaces for EV route optimization

Chapter 6

Conclusion

6.1 Summary

This project addressed the Energy-Minimizing EVRP using Deep Reinforcement Learning. Our contributions include: (1) Comprehensive MILP formulation with multi-trip capability and physics-based energy consumption, (2) Baseline implementations (OR-Tools, Hill Climbing, SA, ACO), (3) Attention-based DRL model achieving competitive performance with fast inference, (4) Interactive dashboard with OpenStreetMap integration.

6.2 Key Findings

The DRL approach generalizes well to unseen instances, achieves significantly lower inference time than iterative methods, effectively captures customer relationships through attention, and provides realistic consumption estimates via physics-based modeling.

6.3 Future Work

Extensions include: larger instances (50+ customers), dynamic scenarios with real-time traffic, multi-depot problems, heterogeneous fleets with varying EV types, and joint optimization of routing and charging schedules.

Appendix A

Code Implementation

A.1 OR-Tools Solver

Listing A.1: Google OR-Tools CVRP solver

```
1 from ortools.constraint_solver import pywrapcp, routing_enums_pb2
2
3 def main(data, max_trip=1):
4     manager = pywrapcp.RoutingIndexManager(
5         len(data["demands"]), data["num_vehicles"] * max_trip, data["depot"])
6     routing = pywrapcp.RoutingModel(manager)
7
8     def distance_callback(from_idx, to_idx, vehicle_id):
9         from_node, to_node = manager.IndexToNode(from_idx), manager.IndexToNode(to_idx)
10        return (data["base_cost_matrix"][from_node][to_node] +
11              data["preference_matrix"][vehicle_id % data["num_vehicles"]][from_node])
12
13    search_params = pywrapcp.DefaultRoutingSearchParameters()
14    search_params.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.AUTOMATIC
15    return routing.SolveWithParameters(search_params)
```

A.2 Physics Constants

Listing A.2: EV energy parameters

```
1 PHYSICS = {
2     'air_density': 1.225, 'rolling_resistance': 0.008, 'gravity': 9.81,
3     'regen_efficiency': 0.8, 'motor_efficiency': 0.9, 'battery_efficiency': 0.95,
4     'hvac_power': 2, 'aux_power': 0.25
5 }
6 TEMP_EFFICIENCY = {-20: 0.55, -10: 0.65, 0: 0.75, 10: 0.88, 20: 1.0, 30: 0.97, 40: 0.88}
```

Bibliography

- [1] Kool, W., van Hoof, H., & Welling, M. (2019). Attention, Learn to Solve Routing Problems! *ICLR*.
- [2] Project OSRM. <http://project-osrm.org/>
- [3] Google OR-Tools. <https://developers.google.com/optimization>
- [4] CVRPLIB. <http://vrp.atd-lab.inf.puc-rio.br/>